The Sidekiq API provides many other features to understand your Sidekiq process.[13] You can grab counts of jobs, failures, and more. Instead of triggering the error catcher, you could also email yourself, or even log statistics to another system.

Note that if you *do* have an existing monitoring system, the Sidekiq Wiki documents several means of integration.[14] Sidekiq Pro[15]—which is modestly priced add-on to the open source Sidekiq—provides a generic integration with statsd,[16] which is a somewhat standard way to get metrics into other systems.

One thing to be aware of is that the Sidekiq API uses Redis and not every Redis command is free.

Learn How to Understand the Cost of Calling Sidekiq API Methods

Sidekiq' API ultimately makes one or more calls to Redis, and these calls won't always return instantly. If you are going to build up a collection of scripts to monitor Sidekiq using its API, you'll need to know how to gauge the cost of those calls. Fortunately, Sidekiq's source code is easy to follow and Redis' documentation is excellent in this regard.

The vast majority of the Sidekiq API implementation is in `lib/sidekiq/api.rb` in Sidekiq's source code.[17] You'll note that each public method makes one or more Redis calls. Usually, a variable named `conn` or `pipeline` is an instance of a Redis wrapper and methods called on those variables are Redis commands. For example, the call to `Sidekiq::ProcessSet#size` that we saw above ultimately calls the Redis command `SCARD`.

Once you know the Redis command, you can consult Redis' documentation to understand the performance implications of that command being called. For example, the documentation for `SCARD`,[18] documents the *complexity* of the command as O(1). This is *Big O Notation* and tells you how expensive it is to make that call.[19] O(1) means that `SCARD` takes the same amount of time to complete, regardless of the contents of Redis (more or less). That means it has